VisionDB CRAFT

STABLE EDITION / VERSION 3.6.0

Complete documentation

VisionDB Craft Stable: A-Z product, operations and technical reference

Prepare → Annotate → Review → Export

A detailed reference for end users, dataset reviewers, Windows deployment operators and software maintainers. Covers the application lifecycle from the Setup wizard and media import to annotation quality, data export, architecture and release acceptance.

How to use this volume

Start with chapters 04-07 (pages 4-8) to set up and launch the application. Labellers and reviewers continue with chapters 08-22 (pages 10-24) and the export chapters; maintainers use the reference chapters near the end. Troubleshooting and the glossary close the volume. This is the Stable edition: it has no auto annotation, AI annotation, model downloads or GPU requirement.

SOURCE SNAPSHOT: 19 OCTOBER 2026

Content is based on the inspected 3.6.0 source packages. Both branded volumes share the same technical content; logos, headings and colour systems follow their respective brands. Version 3.6.0 changes how the application is set up and launched (Setup wizard, quick-launch EXE, optional standalone EXE and installer); the editing functions are unchanged.

Release status: source and setup scripts are provided. Compiled Windows EXEs, clean-machine GUI acceptance are not validated by this documentation.

NAVIGATION / READING GUIDE

Contents

Click a topic to jump to its chapter. PDF bookmarks provide the same navigation.

03 Product scope and edition boundaries
04 Readiness and system requirements
05 Setup wizard: step by step
06 Dependencies and runtime readiness
07 Launcher, standalone EXE and installer
08 Directories, portability and backups
09 Interface, navigation and themes
10 Open images and labelled datasets
11 Projects, recovery and session lifecycle
12 Transform geometry correctly
13 Augmentation controls and data meaning
14 Recipes, seeds and generated variants
15 Resize, previews and memory usage
16 Classes and annotation policy
17 Manual classification and boxes
18 Polygons, masks and segmentation
19 Pose annotations and skeleton compatibility
20 Autosave, History and reset semantics
21 Dataset validation and statistics
22 Train/validation/test splits and leakage
23 Export formats and task filtering
24 Export quality assurance and examples
25 Privacy, network access and licences
26 Architecture and module ownership
27 Backend reference: session and image edits
28 Backend reference: labels and datasets
29 Annotation and dataset data model
30 Configuration, UI state and events
31 Build pipeline and source-package checks
32 Maintenance, migration and extension
33 Troubleshooting setup and startup
34 Release acceptance
35 Glossary, references and support record

Evidence convention: source references name inspected local implementation files. Recommended practices are guidance, not claims that the application enforces them. Automated tests, simulated tests and real deployment checks are distinguished throughout.

STABLE EDITION

Product scope and edition boundaries

VisionDB Craft Stable is an image and computer-vision dataset workbench. It combines image preparation, manual labelling, dataset inspection and export. It is not a training platform, annotation workforce service, model registry server or cloud collaboration system.

What Stable provides

Stable contains three modes: Transform, Augment and manual Annotate, with shared Resize, Export, History, project persistence and Dataset tools. It has no learned-model workflows: no auto annotation, no AI annotation, no model downloads and no GPU requirement. Everything runs locally on the CPU.

Two brands, one documented workflow

This volume documents VisionDB Craft with its Original Light and Original Dark themes. A separately packaged h.ai-branded build follows the same procedures with its own colours, two extra themes, and folder and executable names that start with hai-.

Boundary	Consequence
Edition selection	Fixed at build time; a theme or environment setting cannot change the edition.
Assisted labelling	Stable has none: every label is created or reviewed by a person.
Training	Export data to a separate training workflow; this app does not train your final model.
Team operations	No documented hosted access-control, collaborative editing or central deployment layer.

Practical selection

Use Stable when you want manual labelling or image preparation with few dependencies. A separate Plus edition exists for assisted labelling; it has its own documentation volume. Either way, annotation quality needs human review.

Source reference: imagecraft/edition.py; tools/stage_edition.py; imagecraft/ui/js/mode_manager.js

WINDOWS DEPLOYMENT

Readiness and system requirements

Version 3.6.0 is delivered as source plus a graphical Setup wizard. A source ZIP is not an application installer: Setup builds a private Python environment and a small launcher on your PC. Plan the first setup on a writable local folder with a short path before using production data.

Baseline environment

The supported route targets Windows 10/11 x64 with Python 3.12. If Python 3.12 is missing, Setup.cmd installs Python 3.12.10 for your user account (the installer's signature is checked and no administrator rights are needed). The native window uses pywebview with Microsoft Edge WebView2, which is part of Windows 11. Setup checks for it and installs it when it is missing; Windows may ask for permission.

Space, network and time

Stable's install is small: about 1.5 GB of free disk space and a few minutes. The optional standalone EXE needs about 1.2 GB more and the optional installer about 0.6 GB. The wizard shows the space needed for your selection next to the space available. Downloads need upstream connectivity; the application is not supplied with an offline dependency mirror.

Component	Needed when
Python 3.12	Running Setup. Setup.cmd installs it for you when it is missing.
WebView2 Runtime	Displaying the Windows native application window. Setup installs it when it is missing.
Internet access	Installing libraries during Setup. After setup the application does not need the internet.
Inno Setup 6	Only for the optional Windows installer wizard.
Node.js 18+	Exporting shared source into Stable when tutorial data is processed; not needed to run an already exported app.

The PC check

The first screen of the wizard checks Windows 10/11 64-bit, Python 3.12, whether the folder is writable and sensible, free disk space, WebView2, and whether an install already exists. Only a red failure blocks the Start setup button. Yellow items are advice, such as a folder inside OneDrive, a very long path or a folder under Program Files.

Before starting

Keep original images and labels backed up. Extract the complete ZIP into a new folder with a short path such as C:\VisionDB, rather than running files inside the archive viewer. Record which brand, edition and version you extracted so that later logs can be matched to the installation.

Not a hardware guarantee

No measured minimum RAM or speed has been established for this release. Stable needs no GPU and has no model downloads; all operations run on the CPU.

Source reference: setup_app.py; Setup.cmd; tools/bootstrap.ps1; pyproject.toml

WINDOWS DEPLOYMENT

Setup wizard: step by step

Double-click Setup.cmd in the extracted folder. It starts tools\bootstrap.ps1, which finds Python 3.12 (installing it when missing) and opens the setup window. One window carries the whole process: PC check, options, progress and result. The same engine also runs without a window for unattended use.

1. Extract the zip into a new folder with a short path such as C:\VisionDB, then double-click Setup.cmd. If Windows shows a security prompt, choose to run it.
2. If Python 3.12 is missing, a console shows its download and installation. Nothing has to be started again: the setup window opens by itself afterwards.
3. Read the Your PC panel. Fix any red item; yellow items are advice only.
4. Leave the recommended options ticked and press Start setup. The Start setup button sits in a footer that is always visible. The window scales to your screen size and display scaling, and the content scrolls when the screen is very small.
5. Watch the progress page: each step shows a tick or an arrow, a bar shows overall progress and Show details reveals the live log. You may leave it running. Cancel stops cleanly and keeps what was already downloaded.
6. When "VisionDB Craft Stable is ready" appears, press Open VisionDB Craft Stable, or use the Desktop shortcut or the launcher EXE.

Option	What it does	Default
Install the app	Creates a private .venv inside this folder with OpenCV, NumPy, pywebview and PyYAML.	On
Quick-launch EXE	A tiny "VisionDB Craft Stable.exe" that opens the app from the .venv. Ready in under a minute.	On
Desktop / Start Menu shortcut	Shortcuts that point at the launcher.	On
Standalone EXE with everything inside	The large PyInstaller bundle with Python and every library. Optional.	Off
Windows installer for the standalone build	An Inno Setup wizard; offered when Inno Setup 6 is installed.	Off
Clean reinstall	Deletes .venv and installs everything again (repair).	Off

Python bootstrap

Setup.cmd looks for Python 3.12 through the py launcher and the usual install locations. If it is missing, it downloads Python 3.12.10 from python.org, checks the installer's Authenticode signature and installs it for the current user. If that fails it tries the Windows Package Manager (winget) with the per-user Python 3.12 package. If both fail it prints where to download Python and asks you to run Setup.cmd again. If tkinter (needed for the window) is unavailable, or unattended flags are given, the same steps run in the console instead.

Re-running and repair

Setup is safe to run again at any time. It stores a fingerprint of the requirements (project metadata) in .venv\visiondb-setup.json. When nothing has changed it skips every download and only re-verifies the install, which takes seconds. Close the app before re-running Setup. Tick Clean reinstall to delete .venv and start over, for example after moving the folder or after a failed install.

Command line (optional)

Setup.cmd --yes does the recommended setup without questions. Flags are passed through Setup.cmd. Technical output goes to setup.log; setup-status.json and Setup-Result.txt record how the run ended.

Flag	Effect
--yes	No questions: do the recommended setup plus any other flags.
--standalone	Also build the optional standalone EXE.
--installer	With --standalone: also build the Inno Setup installer.
--no-launcher / --no-shortcuts	Skip the quick-launch EXE or the shortcuts.
--no-deps	Do not install libraries.
--reinstall	Delete .venv and install everything again.
--console	Ask the questions in the console instead of the window.
--check	Print the PC check as JSON and stop.
--dry-run	Show the steps and commands without changing anything.

Source reference: Setup.cmd; tools/bootstrap.ps1; setup_gui.py; setup_app.py

RUNTIME

Dependencies and runtime readiness

Libraries provide the code that runs the application. Setup installs them into a private environment inside the application folder so that nothing is changed system-wide.

Core environment

Core requirements include OpenCV contrib, NumPy, pywebview and PyYAML. Setup creates .venv, installs the package into that environment and checks it with pip check and an import test. The build extra supplies PyInstaller for the optional launcher and standalone build. Keep each edition environment independent rather than copying a .venv from a different machine.

Readiness layer	How to interpret it
Dependencies installed	The required Python modules can be found and imported; it does not prove that your workflow works.
Launcher verified	The launcher EXE found the .venv and launch.py (its --check passed).
Workflow validated	A real test image was opened, edited, saved and exported as intended.
Dataset accepted	A person reviewed labels and confirmed the exported training data.

CPU only

Stable has no GPU, model-checkpoint or AI-library requirement. Every operation runs on the CPU, and no NVIDIA driver is needed.

Troubleshooting strategy

Read the message shown in the setup window together with the final error in setup.log. The wizard turns common failures (file in use, full disk, long path, interrupted download) into a plain sentence and a Try again button; files already downloaded are reused. Resolve the reported step first rather than repeating everything.

Source reference: pyproject.toml; setup_app.py

WINDOWS BUILD

Launcher, standalone EXE and installer

Setup can produce three different Windows artifacts. Only the first is created by default; the other two are optional extras for distributing to PCs without a setup.

Artifact	What it is	Default
Quick-launch EXE	A small program named "VisionDB Craft Stable.exe" that contains no application code. It opens launch.py with the .venv.	On
Standalone EXE	A PyInstaller onedir bundle with Python and every library inside: dist/VisionDB-Craft-Stable/. Large and slow to build.	Off
Installer wizard	An Inno Setup Setup.exe that installs the standalone folder: dist/VisionDB-Craft-Stable-Setup.exe.	Off

Quick-launch EXE

tools/make_launcher.py builds a one-file PyInstaller program in well under a minute. It carries the application icon and version information, finds .venv\Scripts\pythonw.exe and launch.py next to itself and starts the app without a console window. It must stay in the application folder. If the environment or launch.py is missing it offers to run Setup. If the app exits within about five seconds with an error it shows a message instead of failing silently. Its --check option verifies the layout without starting the app; Setup runs it after building. The Desktop and Start Menu shortcuts point at this EXE. Some antivirus products distrust unsigned EXEs: allow it, or open the app with Start.cmd, which does the same thing.

`Standalone EXE (optional)`

tools/build_windows.py runs only on Windows. It uses PyInstaller in onedir mode and gives the output a brand/edition-specific name. The EXE needs its adjacent _internal directory; copying just the EXE is not a supported portable release. After PyInstaller reports Build complete, setup still has named copy/check/inventory stages, and a frozen startup check has a 90-second limit. Completion requires a fresh build-result.json and the expected EXE; an old EXE cannot stand in for a successful new attempt. A release ZIP is not created automatically; maintainers request one with --zip.

`Installer wizard (optional)`

If Inno Setup 6 is detected, Setup can wrap the standalone build in an installer wizard. The recipe uses a writable Documents location. If this optional step fails after the standalone build passed its checks, the final summary says so explicitly; without Inno it is skipped with a note. Native Windows installer behaviour remains unverified here.

Output	Location / handling
Quick-launch EXE	In the application folder, next to launch.py and .venv.
Standalone folder	dist/VisionDB-Craft-Stable/ ; keep the EXE and _internal together.
Installer wizard	dist/VisionDB-Craft-Stable-Setup.exe if Inno compilation succeeds.
Completion record	Setup-Result.txt, setup-status.json and setup.log; build-result.json for the standalone build.

Acceptance condition

Before calling a Windows build ready, test launch on the intended Windows installation, open/edit/save/export sample media, check version/branding. Building successfully is necessary but not sufficient for release.

Source reference: setup_app.py; tools/make_launcher.py; tools/launcher_stub.py; tools/build_windows.py; tools/create_installer.py

BOTH EDITIONS

Directories, portability and backups

Keep installation files, editable projects and exported datasets distinguishable.

Location	Purpose
Extracted source folder	Source, Setup.cmd, Start.cmd, .venv after setup, the quick-launch EXE.
Standalone build (optional)	dist/VisionDB-Craft-Stable/ with the EXE, _internal, documentation.
Per-user configuration	Edition-specific settings.json, kept outside the application folder.
Your chosen project/export folders	Saved workspaces, working labels and downstream dataset outputs.

Move a working installation

Close the app first. Copy the complete application folder. The launcher EXE and shortcuts are created by Setup: after moving the folder run Setup again (tick Clean reinstall if the app no longer opens) so that shortcuts point at the new place. Copy projects and related source media separately. Reopen a project and confirm any relink request before deleting an old installation.

Backup procedure

Maintain an original-media backup, versioned project snapshots, reviewed exports and a record of class names/skeleton conventions. Back up configuration when consistent preferences matter.

Avoid a destructive update

Extract a new version into a new folder and verify it using copies. Do not delete the previous folder simply because a build succeeded.

Source reference: imagecraft/paths.py; imagecraft/settings.py; tools/build_windows.py; tools/make_launcher.py

BOTH EDITIONS

Interface, navigation and themes

The mode launcher selects a focused workflow. Shared panels remain available so that exporting or inspecting history does not require abandoning the current media.

Interface area	Main role
Title bar	Window controls, edition label and appearance selection.
Top toolbar	Open/save, mode switching, undo/redo, reset and zoom.
Viewer	Image inspection, drawing, selected geometry and annotation overlays.
Controls sidebar	Task-specific parameters plus shared panels.
Status area	Current state and operation progress.

Appearance by brand

VisionDB Craft uses the Original Light and Original Dark styling. Choosing a theme changes appearance only. Each edition persists its preferences.

Navigate without losing context

Select a mode appropriate to the next task. set_working_mode changes the focus without destroying the open project. Empty-workspace/discard operations are separate actions. Read pending-apply or destructive confirmations rather than assuming every mode change clears work.

Viewer and performance

Fit the image when you lose the drawing area; Ctrl+0 invokes fit. Inspect fine boundaries at higher zoom before accepting annotations. A downscaled preview is for responsiveness; the applied image operation uses full resolution. Zoom is not an output resize.

Efficient workflow

Keep one task active, finish or discard its preview, then move to review/history/export as needed. Use the in-app Shortcuts dialog for context-specific controls. Keys typed while a text field is active may be intended as text rather than drawing commands.

Source reference: imagecraft/ui/index.html; ui/js/app.js; ui/js/titlebar.js; ui/js/mode_manager.js

BOTH EDITIONS

Open images and labelled datasets

Choose an input route that matches what you have. Opening a folder of ordinary images and opening a labelled dataset are intentionally different operations.

1. Use Open Image for a single file, or Open Images Folder for unlabelled media.
2. Use Open Labelled Dataset for an existing classification, YOLO, COCO or Pascal VOC structure understood by the loaders.
3. Review the imported image count, class names and any reported issues before editing.
4. Inspect a sample with known labels and a sample with unusual dimensions. Only then continue with the full collection.

Import behaviour

Dataset loaders translate supported formats into common internal records and Annotation objects. Parsed labels are not automatically trustworthy because loading succeeded. Missing images, invalid identifiers and malformed geometry need dataset validation and visual review.

Lazy loading

Dataset ImageItem objects can hold original=None with lazy=True until pixels are needed. Source dimensions are retained as metadata. This reduces immediate loading work but does not remove the need for the original media paths to remain accessible.

Input situation	Recommended route
One photograph to edit	Open Image.
Several unlabelled images	Open Images Folder.
Image-label collection with a supported structure	Open Labelled Dataset.
Previously saved editing workspace	Open Project, then relink media if requested.

Check naming and paths

Choose stable media locations and avoid ambiguous duplicate filenames when relinking. A dataset folder is not necessarily recognised just because it contains a label text file; use a supported layout and inspect the resulting summary. Preserve the incoming dataset as a read-only reference copy when possible.

Source reference: imagecraft/dataset/loaders.py; imagecraft/engine.py open_* methods; models/image_item.py

BOTH EDITIONS

Projects, recovery and session lifecycle

A project preserves resumable editing state. Exported training labels serve another purpose. Save a project at meaningful milestones even when working annotation autosave is enabled.

What a project aims to retain

save_project serialises the editing session. The corresponding open_project_path restores media relationships, annotations, history and UI state supported by the manifest. The internal .imagecraft identity is intentionally retained after the rebrand.

1. Save the workspace through Save Project before a large batch operation.
2. Use a new named checkpoint after completing a review pass.
3. Close and reopen a test project to verify the media and annotations resolve.
4. If a moved source cannot be found, use relinking and check the correct image content, not only the filename.

Relinking

relink_missing_media asks for a chosen folder and maps missing source paths to matching filenames. This helps after relocation, but matching names can be ambiguous. Keep directory organisation clear and verify a representative set after relinking.

Reset, discard and close

Reset affects editing state rather than deleting your source file. Discard Session unloads current media/work without deleting source files; unsaved session changes can still be lost. Closing a dataset returns to the normal editor. These actions are not substitutes for backup.

Recovery limits

Undo history is bounded, and autosave is not a complete project archive. A changed output folder or a deleted checkpoint cannot be recovered merely by reopening a project. Back up project files, source media and the final export independently. Never test recovery for the first time on the only working copy.

Source reference: imagecraft/engine.py save_project, open_project_path, relink_missing_media, discard_session

BOTH EDITIONS

Transform geometry correctly

Transform applies geometric corrections to the current active image. Its pipeline order matters: perspective, translation, rotation/scale, then crop. Applying another edit is cumulative.

Control reference

Perspective takes four source corners in pixels. Translation uses horizontal/vertical shifts. Rotation and scaling operate around the centre. Crop can use edge trimming or an accepted free-crop rectangle. Full-resolution crop coordinates are sent to the backend; preview scaling is handled separately.

1. Open the Transform practice image or a copy of a representative sample.
2. Correct perspective before selecting the final crop. Confirm that the chosen corners describe the intended region.
3. Apply modest translation/rotation/scale and inspect borders or clipped content.
4. Preview and accept the crop. Check output dimensions and annotations after committing.

Operation	Review question
Perspective	Are straight lines and the intended plane corrected without removing important content?
Rotation / scale	Are objects clipped or artificially enlarged beyond useful detail?
Crop	Does every remaining label describe an object still visible in the crop?
Repeat Apply	Am I intentionally applying another cumulative transformation?

Annotation alignment

Geometric operations also require label geometry to follow the image. The code contains annotation-aware mapping for supported annotation types. That implementation is not a guarantee that every complex shape remains suitable after a severe warp; inspect boundary samples and validate the exported result.

Worked example

For a tilted equipment label, adjust perspective using its corners, straighten with a small rotation if needed, then crop. Review the changed object box or mask. Save a project checkpoint and a small exported sample before processing the next images.

Source reference: models/transformation.py run_transform_pipeline; dataset/geometry.py; ui/js/panels/transform.js

BOTH EDITIONS

Augmentation controls and data meaning

Augmentation creates variation for a downstream training task. A visually interesting effect is not automatically a useful training example. Preserve the meaning of the label.

Available operation families

The augmentation pipeline includes brightness-like white balance, HSV adjustments, shear, noise, flips, padding, salt-and-pepper noise, Gaussian blur, softness, sharpening, oil-painting and exposure controls. Some names describe effects rather than calibrated camera physics.

Family	Useful review
Colour / exposure	Does the scene remain plausible for the deployment environment?
Flip / shear	Are direction-sensitive labels or keypoints still meaningful?
Noise / blur	Is the object still identifiable at the difficulty level you intend?
Padding	Are image size and label coordinates still consistent?
Stylisation	Does it represent a real variation, or just add unrelated texture?

1. Begin with one representative image and a conservative setting.
2. Compare source and processed views and inspect the changed labels.
3. Apply once, check History and decide whether the change should be repeated.
4. Use controlled recipes for planned dataset variation rather than repeatedly clicking Apply without tracking settings.

Working-image operations versus variants

Applying an effect changes the active working image and creates a history event. Generating variants creates additional dataset examples through a seeded recipe. The two routes serve different purposes and should not be mixed without tracking which images are derived from which sources.

Example and quality boundary

A moderate brightness variation may represent site lighting. A horizontal flip may be unsuitable if a class describes left/right actions or readable signage. Keep source-derived families in a controlled split policy; do not let close variants leak from training into evaluation sets.

Source reference: models/transformation.py run_augment_pipeline; dataset/recipes.py; ui/js/panels/augment.js

BOTH EDITIONS

Recipes, seeds and generated variants

A recipe defines a controlled family of augmentation settings. A seed makes the random choices reproducible for the same data and implementation, rather than proving that a generated dataset is good.

Backend contract

get_augmentation_recipes returns the available recipes. generate_variants accepts recipe_id, copies, seed, optional custom_ranges and annotation_aware. The legacy generate_dataset_variants entry point delegates to the unified route. Keep a record of these inputs when creating a dataset revision.

1. Inspect the recipe and its ranges using the app controls.
2. Choose a small copy count and record the seed.
3. Generate variants from representative inputs and inspect both images and labels.
4. Validate the expanded dataset and review split membership before export.
5. Only increase the copy count when the variation is useful and the storage/memory cost is acceptable.

Parameter	Interpretation
recipe_id	Selects the registered operation/range recipe.
copies	Requested generated variations per input under the chosen workflow.
seed	Controls deterministic random choices; record it with the dataset revision.
custom_ranges	Overrides supported recipe ranges where the UI/backend exposes them.
annotation_aware	Requests supported geometry updates alongside the image variation.

Do not equate volume with diversity

Ten near-identical variants do not necessarily add ten independent examples. Review class balance and the source-image distribution as well as file count. Real deployment diversity still depends on the original capture conditions and subjects.

Reproducibility record

Store the app version, source dataset identity, class mapping, recipe, custom range values, seed, copy count and export format. A code or dependency change can change results even with the same seed; compare a small known sample after updating.

Source reference: imagecraft/engine.py get_augmentation_recipes, generate_variants; dataset/recipes.py

BOTH EDITIONS

Resize, previews and memory usage

The displayed preview, active full-resolution image and exported output are related but not identical. Distinguish display zoom from an actual change in pixel dimensions.

Preview versus Apply

The preview route works on a downscaled copy sized by the active performance profile to keep controls responsive. Apply processes the active image at full resolution and records the operation. A fast preview therefore does not establish the memory or time cost of a full batch.

Resize procedure

Use the Resize panel, set the intended dimensions and review the result on a small sample. Check the saved dimensions by reopening an exported image. Inspect boxes, masks and keypoints after resizing; a correct-looking preview is only one part of the check.

Action	What changes
Zoom / Fit	Viewer display only.
Preview	Temporary representation for inspection.
Apply resize	Actual working image dimensions and corresponding supported labels.
Export	Written image/label files according to the selected route.

Memory management

Large originals, processed arrays, previews and undo records can all contribute to memory pressure. Lazy dataset loading helps with unmodified sources. A long history or large batch can still be expensive. Save a project before large operations and test representative samples first.

Performance profile nuance

The settings use internal low/medium profile identifiers with UI naming documented in the code. Read the active UI description rather than interpreting the internal identifier as a benchmark. No measured speed or memory guarantee is supplied for this version.

When memory is tight

Reduce the working batch size, avoid unnecessary repeated full-resolution edits, and save and reopen the project if appropriate. Closing other applications also helps.

Source reference: imagecraft/engine.py preview, apply, get_preview; imagecraft/io.py downscale; settings.py

BOTH EDITIONS

Classes and annotation policy

A class list is part of the dataset contract. Consistent names and class identifiers matter more than the convenience of adding a label during one image.

Class operations

The backend supports adding, renaming and importing class names. Import accepts classes.txt, dataset YAML or COCO JSON through the class-import workflow. Removing a class can reassign affected annotations or remove them, depending on the explicit options.

1. Write a short labelling policy before starting: class names, boundaries, occlusion rules and what not to label.
2. Import or create the agreed list and test it on a sample.
3. Use one-class mode only when the intended task genuinely assigns the same class to repeated objects.
4. Before deleting/reindexing a class, save a project copy and inspect affected images.
5. Verify names and indices in the exported configuration.

Decision	Example
Definition	A helmet class means a visible helmet, not a person likely to be wearing one.
Occlusion	Agree whether to box only the visible region or the inferred full extent.
Ambiguous cases	Flag for human review rather than inventing a new near-duplicate class.
Class change	Renaming a display label and deleting all annotations of that class are different operations.

History interaction

Class reindexing must remain consistent with stored annotations and undo/redo records. The engine includes traversal of live/history annotation sets for class updates. Nevertheless, test the resulting dataset before distributing a class-map revision.

Recommended record

Keep the class list with the dataset revision and annotate the reason for any class merge/removal. If another project uses a different order, reconcile the mapping deliberately instead of copying label files under the assumption that class 0 always means the same object.

Source reference: imagecraft/engine.py add/import/rename/delete_annotation_class; dataset/models.py

BOTH EDITIONS

Manual classification and boxes

Classification labels an image; object detection labels the location of an instance. Do not create dummy boxes merely to represent an image-level class.

Classification workflow

Choose the classification task and the intended class. Review whether the dataset policy permits multiple labels per image. The internal classification annotation records a class without inventing spatial object geometry. Export it using the classification route rather than a detection export.

Box workflow

Select the box drawing tool and class, draw the object boundary, then inspect it at an appropriate zoom. Use selection/editing controls to move or resize where supported. A second object needs its own instance annotation even when it has the same class.

1. Open a representative image and choose the task.
2. Select the class before drawing or assigning the label.
3. Inspect the annotation list and remove accidental duplicates.
4. Review small/occluded objects according to the project policy.
5. Save working labels and test a task-appropriate export.

Check	Detection-specific question
Tightness	Does the box fit the agreed object extent?
Completeness	Are all required instances labelled, including difficult ones?
Class accuracy	Does the selected class match the actual object?
Duplicates	Has the same instance been labelled twice?
Boundary cases	Are partly visible objects handled consistently?

Quality review example

For a site photo with two helmets and a toolbox, review the class of each object separately and check the partially hidden helmet against the occlusion rule. A correct class count does not prove the boxes are correctly positioned.

Drawing controls

Drawing shortcuts are context-sensitive. Keep Annotate active and avoid keyboard focus in a text field when invoking drawing keys. Use the in-app Shortcuts guide for the exact controls supplied by the running version.

Source reference: imagecraft/engine.py create_annotation, update_annotation_geometry; ui/js/panels/annotation.js

BOTH EDITIONS

Polygons, masks and segmentation

Segmentation represents an object region rather than only its rectangular extent. The application handles polygon and mask representations, but output formats can preserve different information.

Polygon versus mask

A polygon stores boundary vertices. A bitmap mask is represented internally through mask_rle data. A traced polygon export is not automatically equivalent to the original bitmap around small holes, fine boundaries or disconnected components. Choose the representation the downstream task expects.

1. Choose the segmentation task/tool and correct class.
2. Create the object boundary or mask using the available manual tools.
3. Zoom into edges and inspect thin structures, holes and overlapping objects.
4. Review the annotation list for accidental duplicate regions.
5. Export a sample to the chosen segmentation format and overlay it independently if possible.

Situation	Review focus
Thin object	Small features may disappear during tracing or resizing.
Occlusion	Label according to the visible/inferred-boundary policy.
Multiple components	Confirm whether the chosen representation preserves the needed separation.
Geometric augmentation	Check the entire transformed boundary, not only the enclosing box.

Shared geometry

The Annotation model carries polygon points or a mask RLE in addition to bounding information. Geometry helpers map supported shapes through image operations; mask helpers encode/decode and derive bounds. A common representation lets every region follow the same review and export route.

Export choice

COCO instance segmentation is offered for preserving masks; YOLO segmentation uses polygon-style labels and can trace masks. Inspect the result with the receiving framework before using it for training. Do not assume a box export includes the underlying segmentation boundary.

Source reference: dataset/models.py; dataset/masks.py; dataset/geometry.py; dataset/exporters.py

STABLE EDITION

Pose annotations and skeleton compatibility

A pose annotation includes ordered keypoints and their visibility information. The order and skeleton convention must match the downstream training configuration.

Representation

The Annotation model includes keypoints, pose_schema, pose_workflow and pose_has_bbox. Individual points carry coordinates and visibility information. update_pose_keypoint updates a landmark without treating the entire structure as an ordinary box.

1. Choose the supported pose workflow or import a compatible pose dataset.
2. Inspect the keypoint order and skeleton definition before labelling many images.
3. Move landmarks and set visibility according to the task convention.
4. Check left/right consistency and the person/object association.
5. Export only to a pose format compatible with the actual schema.

Risk	What to check
Reversed left/right	Use the subject convention consistently, not whichever side appears on screen.
Invisible points	Do not present an inferred point as clearly visible unless policy allows it.
Wrong skeleton	COCO-17 and other landmark layouts cannot be exchanged by simply renaming files.
Missing person box	Some formats/workflows require a valid box as well as keypoints.

Example review

For a partly occluded worker, verify joint positions, visibility and the correct person association. A high confidence on a visible wrist does not validate a hidden hip. Inspect the exported keypoint order and any required skeleton metadata before a training run.

Source reference: dataset/pose.py; dataset/models.py; engine.py update_pose_keypoint

BOTH EDITIONS

Autosave, History and reset semantics

Saving images, saving working annotations, saving a project and exporting training data are separate actions. Know which artifact you need before treating work as safely preserved.

Action	Purpose
Working annotation save / autosave	Persist editable working labels to the configured location.
Save image	Write current processed image output through the selected save path.
Save modified images	Write images considered modified in the session.
Save Session	Write every loaded image, including untouched images.
Save Project	Preserve a resumable workspace, rather than just training files.

History capacity

The engine defines MAX_OP_HISTORY=20. As older entries are removed, the base is marked as truncated and the History view can show Earlier state rather than Original. Undo is not an unlimited archive of every change since import.

Preview and commit

Slider previews do not create the same persistent operation history as Apply.

1. Save a project before a large batch edit.
2. Review the History parameters to confirm what was applied.
3. Use Undo/Redo or jump to a visible history state only when that is the intended revision.
4. After a class or geometry correction, inspect working labels and the final export.

Reset and discard

Reset can restore the working image state; Discard Session unloads work. Neither should be interpreted as a recoverable backup action. Once history has been truncated or a session discarded, use a saved project/original dataset rather than assuming the original editing chain remains available.

Source reference: imagecraft/engine.py MAX_OP_HISTORY, save_session, save_all, undo/redo, reset_current; settings.py

BOTH EDITIONS

Dataset validation and statistics

The Dataset panel helps reveal structural problems. It does not prove that every annotation is semantically correct. Combine automated validation with representative visual review.

Implemented checks

The validator reports issues including missing media, malformed/imported issues, invalid class identifiers, out-of-range or zero-area geometry, duplicate annotations, tiny boxes and duplicate images. Duplicate content across splits is treated as leakage. Different annotation types have their own geometry constraints in the common model.

Statistics

Dataset statistics include counts by classes and splits. Review image counts and object counts separately; a dataset can contain many images while a rare class has very few instances. Statistics are descriptive rather than a guarantee of balanced training performance.

1. Open the labelled dataset and run validation.
2. Read error codes and the associated paths.
3. Investigate a representative example of each recurring issue.
4. Fix the source or working annotations deliberately and rerun the relevant check.
5. Inspect any cleaned/fixed export to understand which invalid annotations were omitted.

Finding	Response
Invalid class ID	Reconcile label identifiers with the class list.
Out-of-range geometry	Inspect the image/label dimensions and previous transforms.
Duplicate annotation	Keep the intended instance and remove the accidental duplicate.
Split leakage	Reconsider split assignment for duplicate or related images.
Tiny box	Check whether the object is meaningful, correctly located and consistently labelled.

Caution on cleaned export

export_fixed_dataset can omit invalid annotations. A cleaner validation report after omission does not mean the missing objects were correctly relabelled. Track exclusions so that a data-cleaning step does not silently change the task coverage.

Source reference: dataset/analysis.py; dataset/models.py is_normalized; engine.py validate_current_dataset, export_fixed_dataset

BOTH EDITIONS

Train/validation/test splits and leakage

Train, validation and test partitions should support an honest evaluation of the intended task. Random reproducibility and independence are different properties.

Implemented split mechanism

split_current_dataset defaults to 70/20/10 with seed 42. The dataset module uses deterministic, class-aware assignment and capacity accounting. This can help distribute classes, but it is not a business-aware grouping system that automatically knows which photos belong to one site, person, capture session or asset.

1. Define the unit that must remain independent, such as site, subject or capture session.
2. Remove/reconcile exact duplicates and identify related near-duplicates.
3. Choose ratios and a seed suited to the dataset size.
4. Review actual class and split counts after assignment.
5. Export and verify that source-derived families do not straddle evaluation boundaries.

Leakage example	Why it misleads
Identical image in train and test	Evaluation may measure memorisation rather than generalisation.
Variants of one source across splits	Small appearance changes can preserve the same scene information.
Same project capture series across splits	Closely related views can be easier than a new deployment site.
Rare class absent from test	The reported evaluation cannot establish quality for that class.

Recommended policy

Where group-level independence matters, organise/group the source data deliberately before assignment, or use an external split procedure and reimport the result. The in-app exact/perceptual duplicate checks are useful evidence, not a complete detector of every relationship.

Record the decision

Store the seed, ratios, source dataset revision and grouping rationale. When you add data or regenerate variants, re-check independence rather than assuming the old split policy still holds automatically.

Source reference: dataset/analysis.py assign_splits, validate_dataset; engine.py split_current_dataset

BOTH EDITIONS

Export formats and task filtering

An export must match the downstream learning task. The UI offers only formats valid for the selected task and available annotation structure.

Task	Typical offered outputs	Important boundary
Detection	YOLO Detection TXT; COCO Instances; Pascal VOC XML	Boxes do not carry the underlying mask or keypoint structure.
Segmentation	COCO instance segmentation; YOLO segmentation	YOLO uses polygons/tracing, so review conversion detail.
Classification	Class folders	Image-level labels are not object boxes.
Pose	COCO Keypoints; compatible YOLO Pose options	Skeleton, keypoint order and box requirements must match.

1. Select Export, intended image scope and annotation task.
2. Choose a valid format from the actual dropdown.
3. Choose a destination that distinguishes the new revision.
4. Export a sample and inspect the reported saved/excluded counts.
5. Validate that sample in the receiving training tool before exporting everything.

Mixed-task projects

A workspace can contain different annotation types, but a training export should be coherent. export_annotations filters for compatible geometry and can report annotations from other tasks that were excluded. Those exclusions are intentional and need to be understood rather than interpreted as missing-file corruption.

Working files versus training labels

Internal working annotations preserve application metadata, including review/provenance details. Standard training formats deliberately omit application-only metadata. Keep the project/working copy if those details are needed later for audit or continued editing.

Naming

Use a clear export root for each dataset revision, task and class schema. File names alone are not enough to establish that image dimensions, class order and the annotation representation match the receiving framework.

Source reference: ui/js/panels/export.js; engine.py export_annotations; dataset/exporters.py

BOTH EDITIONS

Export quality assurance and examples

The last useful check is against written output, not only what appeared on screen. Build a small repeatable export test set containing normal and difficult examples.

Sample contents

Include an image with multiple objects, a small or occluded object, a non-square image, a transformed image and the annotation representation you rely on. For pose include visibility cases; for segmentation include thin or disconnected regions where relevant.

1. Save a project snapshot before final export.
2. Export the small test set to a new folder.
3. Reopen/overlay labels using an independent reader or the target training environment where possible.
4. Check dimensions, class identifiers, geometry and image-label pairing.
5. Review excluded/invalid-label counts and document deliberate omissions.
6. Export the complete dataset and check counts/statistics again.

Example failure	Likely investigation
Boxes shift after export	Coordinate convention, image dimensions or transform mapping.
Correct name, wrong class	Class index ordering/configuration mismatch.
Mask loses detail	Polygon tracing or incompatible representation.
Pose joints swapped	Skeleton order or left/right convention.
Images absent	Current/all scope, modified-only save choice or invalid/missing inputs.

Release record

Store app version, source dataset revision, class list, split policy, transform/recipe settings, export task/format and reviewer sign-off. This can be a simple project record; the app does not claim to provide a complete regulated audit trail.

Practical acceptance

Accept the export when the receiving tool reads it correctly, the sample overlays are right, counts are understood and required classes/edge cases are represented. A parseable JSON file by itself is insufficient evidence of a useful training dataset.

Source reference: dataset/exporters.py; dataset/analysis.py; engine.py export_annotations, save_all, save_session

PRIVACY AND LICENCES

Privacy, network access and licences

Stable processes your images locally and has no hosted-model or cloud component. The application does not intentionally send your images anywhere. That design statement is narrower than a complete security certification.

Network boundaries

Setup contacts package and vendor services while it runs: PyPI for libraries, python.org for Python and Microsoft for WebView2 when they are missing. After setup the application does not need the internet. There is no supplied egress audit or independently verified network-isolation certification.

Data handling

Keep sensitive media, working labels, projects and exports in locations approved by your organisation. The app does not document encryption at rest, role-based access control or a central retention policy. Those responsibilities remain with the deployment environment and operating procedures.

Artifact	Review separately
Application code	Existing MIT licence and preserved notices.
Python dependencies	Each library licence and binary redistribution obligations.
Training images and labels	Rights, consent and intended downstream use.
Distribution package	Bundled notices and an actual dependency/version inventory.

Practical deployment record

Record approved input locations, access permissions, backup/retention expectations and who reviews exports. For a restricted or offline environment, prepare and validate dependencies in advance; the supplied ZIP is not a self-contained offline installer.

Security limitation

This documentation describes inspected code paths and current tests. It does not claim penetration testing, signed binaries, malware scanning results or compliance approval that has not actually been performed.

Source reference: pyproject.toml; LICENSE; THIRD_PARTY_NOTICES.md; tools/bootstrap.ps1

DEVELOPER REFERENCE

Architecture and module ownership

The application combines a Python backend with a local HTML/CSS/JavaScript interface displayed through pywebview. The internal imagecraft package name is retained for compatibility after the rebrand.

Request flow

A frontend panel gathers task values and calls a backend method through ui/js/api.js. BackendEngine and the edition feature mixin execute the operation on shared image/dataset state. Responses and custom events drive frontend state/progress updates. This is a desktop bridge, not a documented public HTTP/REST service.

Module group	Responsibility
main.py / launch.py	Native startup and frozen/source self-test entry.
engine.py	Session orchestration, media, history, projects and shared API.
core_features.py	Stable's lifecycle and shared capability surface.
models/	Image/collection state and image processing parameters.
dataset/	Common labels, geometry, loaders, analysis and exporters.
ui/js/panels/	Task controls and user actions.

Edition architecture

edition.py is a fixed descriptor importing the relevant feature class. stage_edition.py produces independent exports; this Stable package contains no Plus implementation modules or controls. Retain this boundary when adding features rather than exposing runtime edition toggles.

Change strategy

Edit shared logic in a master source, run affected tests, export the editions and inspect the actual exported packages. A passing master tree cannot establish that Stable has no dangling Plus import after packaging.

Scope of API pages

The next references document selected methods found in the current source. They are maintainer entry points, not a promise of a versioned external SDK. Confirm signatures and return handling when changing the frontend or building integrations.

Source reference: imagecraft/main.py; engine.py; edition.py; ui/js/api.js; tools/stage_edition.py

DEVELOPER REFERENCE / INTERNAL BRIDGE

Backend reference: session and image edits

BackendEngine orchestrates active media and operation state. Preview and Apply are separate contracts: changing a preview does not imply that a full-resolution result has been committed.

`get_state(self)`

Retrieve the current session/view payload after an operation so the UI can refresh consistently.

`set_working_mode(self, mode: str)`

Change focused workflow while keeping the current project. Unsupported edition modes are rejected.

`save_project(self, ui_state: dict[str, Any] | None=None, target_path: str='')`

Persist a resumable workspace. An empty target path uses the application save-dialog route.

`preview(self, kind: str, params: dict[str, Any])`

Request a temporary downscaled preview for a supported operation kind and parameter dictionary.

`apply(self, kind: str, params: dict[str, Any])`

Apply an operation at full resolution to the current active image and add an undo/history record.

`apply_all(self, kind: str, params: dict[str, Any])`

Apply the selected operation across open images using progress and cancellation support.

Calling convention

Frontend code calls these methods through the local pywebview bridge using call(method, ...args). Positional arguments must match the current signature. Handle rejected bridge calls and inspect the returned payload rather than assuming that a fulfilled JavaScript promise means the requested operation succeeded.

Compatibility boundary

These are selected internal entry points from version 3.6.0, not a supported external network SDK. Check the implementation when changing return structures or building automation. Desktop dialog methods require the native bridge; do not treat a browser-only mock as evidence that a file dialog worked.

Source reference: imagecraft/engine.py; imagecraft/ui/js/api.js

DEVELOPER REFERENCE / INTERNAL BRIDGE

Backend reference: labels and datasets

The shared core converts incoming labels to common Annotation objects. Mutations must preserve class, coordinate, history and export consistency.

`create_annotation(self, annotation_type: str, class_id: int, geometry: dict[str, Any] | None=None)`

Create an annotation of the selected type/class from compatible geometry. Confirm task-specific fields.

`transform_annotation_geometry(self, index: int, geometry: dict[str, Any])`

Move/resize supported spatial geometry on the current image rather than changing only a class label.

`delete_annotation_class(self, class_id: int, reassign_to: int=-1, delete_annotations: bool=False)`

Remove a class with deliberate reassignment or annotation deletion; class IDs need consistent reindexing.

`export_annotations(self, target_format: str='yolo', scope: str='all', task: str='auto')`

Export a coherent task for current/all scope with only compatible annotation geometry.

`validate_current_dataset(self)`

Return the current structural dataset validation report; visual/semantic review is still necessary.

`split_current_dataset(self, train_percent: float=70, val_percent: float=20, test_percent: float=10, seed: int=42)`

Assign train/validation/test proportions with the given seed. Group independence needs a separate policy.

Calling convention

Frontend code calls these methods through the local pywebview bridge using call(method, ...args). Positional arguments must match the current signature. Handle rejected bridge calls and inspect the returned payload rather than assuming that a fulfilled JavaScript promise means the requested operation succeeded.

Compatibility boundary

These are selected internal entry points from version 3.6.0, not a supported external network SDK. Check the implementation when changing return structures or building automation. Desktop dialog methods require the native bridge; do not treat a browser-only mock as evidence that a file dialog worked.

Source reference: imagecraft/engine.py; imagecraft/ui/js/api.js

DEVELOPER REFERENCE

Annotation and dataset data model

The common schema lets every label share validation, history and export. Do not write a new exporter that assumes every Annotation is a bounding box.

Field group	Meaning
class_id / label	Numeric class mapping and optional readable label.
x_center, y_center, width, height	Normalised bounding information for supported spatial annotations.
annotation_type	Distinguishes bbox, classification, polygon/mask or pose workflow data.
polygon / mask_rle	Boundary vertices or encoded bitmap representation.
keypoints / pose_schema / pose_workflow	Ordered landmarks and compatibility metadata.
metadata	Application-only provenance/review fields; standard exporters omit these.

Serialization detail

Annotation.as_dict exposes x, y, w, h and type rather than the dataclass attribute names x_center, y_center, width, height and annotation_type. Preserve this distinction when reading bridge payloads. Classification serialization is not a request to invent spatial labels.

ImageItem

An item retains source pixels or a lazy source path, optional modified pixels, original/modified annotations, split membership and source dimensions. The active property selects the current image, and active_annotations follows its geometry. Copies are used to avoid mutating the stored original annotation objects inadvertently.

DatasetProject

The dataset project records root, format, class names, records, imported issues and task type. DatasetRecord associates an image path with its label path, split, annotations and parse issues. UI state is not the only source of dataset truth.

Implementation caution

Use existing copy/geometry helpers so that masks, keypoints and metadata are not accidentally shared/mutated across history records. Validate geometry through the model rules and then inspect the exported task, especially after coordinate conversions.

Source reference: dataset/models.py Annotation, DatasetRecord, DatasetProject; models/image_item.py

DEVELOPER REFERENCE

Configuration, UI state and events

Persistent settings, frontend view state and the running session are separate state layers. Do not treat a saved theme preference as part of the dataset annotation schema.

Persistent settings

Settings loads JSON, overlays recognised keys on defaults and writes changes immediately. Defaults include theme, sidebar placement/width, performance mode, live preview, export prefix/postfix, recent projects and annotation save/class preferences.

Brand/edition isolation

config_root selects the per-user brand directory plus the lower-case edition. VisionDB accepts VISIONDB_CRAFT_CONFIG with a legacy IMAGECRAFT_CONFIG fallback. (The separate h.ai build adds HAI_VISIONDB_CRAFT_CONFIG before those.) Legacy ImageCraft settings can seed recognised defaults without overwriting the old file.

State	Owner / lifetime
settings.json	Per-user brand/edition preferences across launches.
Frontend state.js	Active view/tool and received backend state during a session.
Project UI data	Resumable workspace view information where saved by the project route.

Bridge events

api.js wraps the pywebview API and raises when a method is unavailable. Backend events are delivered as browser CustomEvents named backend:<event>. onReady waits briefly for bridge injection and permits a plain-browser UI boot path without pretending backend operations exist.

Testing frontend-only behaviour

A mock bridge can establish that a button calls the expected method and updates view state. It cannot establish native-window behaviour or filesystem dialogs. Label tests accordingly and supplement them with actual application checks.

Configuration caution

Settings.load logs invalid JSON/read errors and continues with defaults. Keep a backup before manual edits. Unknown keys are not a supported extension mechanism; add a deliberate default/UI/backend contract when introducing a new setting.

Source reference: settings.py; paths.py; ui/js/state.js; ui/js/api.js

DEVELOPER REFERENCE

Build pipeline and source-package checks

A build recipe should make the resulting package independently usable and make its limitations explicit. Shared-tree tests alone do not prove a clean exported edition.

Export sequence

Edition export strips Plus implementation from Stable. The 3.6.0 source packages ship with the Setup wizard, the launcher tools and the current documentation, so nothing else has to be applied. Optional release ZIP compression is not part of Setup; maintainers request it with --zip on tools/build_windows.py.

Check	Evidence it can establish
Python tests	Backend invariants and exercised workflow contracts.
Frontend syntax/import checks	Parseable JavaScript and valid local imports, not UI appearance.
Independent export startup	The edition resolves its own code/resources.
Frozen --self-test	Packaged backend/resources start on the build host.
Interactive clean-machine test	Native UI, dialogs, runtime prerequisites and actual workflow usability.

Version record

Version 3.6.0 changed how the application is set up and launched, not its editing functions. The setup engine's tests (command ordering, fingerprint skipping, brand detection, the wizard palette for every brand, build-script options) run on any operating system because commands are recorded rather than executed. Native Windows and hardware validation are separate from those source tests.

Reproducibility limit

Core requirements are declared in pyproject.toml with version ranges. Record resolved versions (pip freeze) for controlled distribution. Source hashes record shipped bytes, but the runtime process is not a guarantee of byte-identical binaries across dates.

CI boundary

A frozen Windows build needs a Windows x64 host; Linux cannot produce a Windows EXE. No precompiled Windows EXE is delivered here. Setup logs stage labels, so PyInstaller completion is not confused with completion of the whole setup.

Source reference: tools/stage_edition.py; tools/build_windows.py; UPDATE-REPORT.md

MAINTAINER OPERATIONS

Maintenance, migration and extension

Preserve the shared core, project compatibility and edition boundaries while evolving the app. User-facing branding should not require changing internal project identifiers.

Update procedure

Keep the previous verified folder. Extract the new package separately, run its Setup, test a known sample. Setup's fingerprint stamp lets a repeated run skip unchanged downloads. Compare project re-open, annotation overlays and exports before switching the primary installation.

Adding a core feature

Identify image/annotation state changes, add a shared backend method and corresponding panel controls, update history/serialization/export where needed and document the interaction. Verify Stable independently so that a new shared import does not pull in Plus code.

Change	Checks to add or revisit
New transform	Image pixels, annotation geometry and undo/redo.
New annotation type	Schema, import, edit, validation, project and export.
Brand/theme edit	Resource paths, persisted theme values, contrast and both editions.
Packaging change	Extracted startup, resource collection and clean Windows launch.

Documentation discipline

Update the quick manual for routine usage and this reference for architecture/data contracts. Keep tested, implemented and planned behaviour distinct. If an older README contradicts current code, identify the version and reconcile it rather than copying the stale claim into a new release.

Source reference: edition.py; dataset/models.py; tools/stage_edition.py; paths.py

OPERATIONS RUNBOOK

Troubleshooting setup and startup

Use the failed step and launch mode to narrow a problem. Repeating the entire setup without reading the final message can hide whether the issue is Python, a package, the launcher or a native-window prerequisite.

Symptom	First action
Setup.cmd closes at once or shows Python errors	Read the red text in its window. Install Python 3.12 from python.org if the automatic install failed, then run Setup.cmd again.
"File in use" / Access is denied	Close VisionDB Craft Stable and anything else opened from the folder, then press Try again.
Drive full	Free disk space (see chapter 04) and press Try again.
Path too long	Move the folder next to the drive root, for example C:\VisionDB, and run Setup again.
Download interrupted	Check VPN, proxy and firewall, then press Try again; downloaded files are reused.
pip check reports a conflict	Tick Clean reinstall and run Setup again rather than ignoring the consistency check.
Launcher says it cannot find the app	The EXE must stay beside launch.py and .venv. Run Setup again, or open the app with Start.cmd.
No native window	Check WebView2 and that the folder is complete.
Shortcut opens the wrong place	The folder was moved. Run Setup again so the shortcuts are recreated.
Standalone step fails	Read the last build output in setup.log. Do not use an old dist result as evidence.
Installer wizard not produced	Check Inno Setup 6; an installer failure is reported separately from a working standalone EXE.

Minimal reproduction

Check the final page of the wizard, setup-status.json and Setup-Result.txt. If the console version was used, read the last named step. Choose Cancel (or press Ctrl+C in the console) if necessary and do not start two setups in one folder. Setup --dry-run shows the planned commands without changing anything, and --check prints the PC check.

Source versus launcher versus standalone

A No module named webview error after python launch.py usually means that a global Python was used instead of the .venv created by Setup. Use the launcher, Start.cmd or .\.venv\Scripts\python.exe launch.py. The launch script also hands over to the local environment automatically on Windows when needed. The standalone EXE uses its bundled dependencies and does not use .venv. Repairing the source environment does not retrofit an older standalone EXE; rebuild it.

What to send for support

Include the final message from the wizard, the relevant section of setup.log, setup-status.json, whether dependencies were installed in this folder, and, for a standalone build, whether _internal is adjacent to the EXE. Share a non-sensitive reproducible sample when the failure depends on a file. Never send confidential source media merely to demonstrate that a window failed to open.

Source reference: Setup.cmd; setup_app.py; tools/bootstrap.ps1; tools/launcher_stub.py; main.py; logging_config.py

OPERATIONS RUNBOOK

Release acceptance

Application, runtime and data problems are different failure categories. Diagnose them separately.

Symptom	Next action
The app does not open from the launcher	Run the launcher's --check, then chapter 33. Setup can be repeated safely.
An old standalone EXE misbehaves	Rebuild it: repairing the source environment does not retrofit a frozen EXE.
Export looks wrong	Follow the sample-export checks in chapter 24 before changing the dataset.
Boxes or masks look shifted after Transform/Resize	Compare the exported sample against the original and check the order of operations.

Release acceptance sequence

On a clean intended Windows machine, run Setup with the recommended options, open the app from the shortcut and from the launcher, run core edit, annotate, save and export on your own images, and, if you distribute the standalone build, verify its own startup. Move and reopen a test project and confirm that media and annotations resolve.

Known evidence limits

The delivered verification covers source regression, syntax and the setup engine's command sequencing. It does not establish clean Windows installer behaviour or native GUI acceptance. Those remain explicit outstanding checks.

Pass criteria

Record actual results rather than a single generic pass. At minimum, the intended user can install, launch, complete a representative workflow and export correct data. Retain failure details and the exact environment when a criterion is not met.

Scope control

A source package with build scripts should not be labelled a prebuilt Windows release. Keep that distinction in the user-facing release notes.

Source reference: UPDATE-REPORT.md; launch.py; setup_app.py

REFERENCE

Glossary, references and support record

Use these definitions when discussing the app with reviewers or maintainers. The source references in each chapter identify inspected implementation files, not a claim of external certification.

Term	Meaning in this documentation
Annotation	An accepted/manual image label, box, region or keypoint structure.
Runtime	The Python/native libraries that execute the application's code.
Source package	Editable application files and scripts; no implied bundled EXE.
Frozen / onedir	A built EXE plus adjacent bundled runtime files.
Project	Resumable workspace using retained .imagecraft compatibility.
Export	Task-specific output for downstream data/training use.
Seed	Input controlling reproducible random choices under the same implementation.
Split leakage	Related/duplicate information crossing evaluation boundaries.
RLE	Run-length representation used for bitmap-mask data.
Launcher	The small quick-launch EXE that opens launch.py with the .venv; it contains no application code.
Standalone EXE	The optional PyInstaller bundle that contains Python and every library.

Version and authoritative source

This volume documents the 3.6.0 source/update package inspected on 19 October 2026. Runtime labels/signatures can change in later builds. The current code, setup_app.py, PACKAGE-STATUS.json and UPDATE-REPORT.md should be checked together when an older bundled guide disagrees.

Support record template

Record: brand; edition; app version; source or frozen launch; Windows/Python/runtime versions; input task/format; reproduction steps; expected versus actual result; exact error; relevant setup.log and setup-status.json; whether originals are backed up.

Final operational boundary

The user manual is the short how-to companion. This reference explains procedures, state/data contracts, architecture and acceptance. It does not replace a completed Windows release test or an organisational data-handling policy.

Source reference: PACKAGE-STATUS.json; UPDATE-REPORT.md; code references throughout this volume